

Project Title: Enhancing LLM Code Maintainability via Future-Task Prompting

Participants: Ariel Zoizner (209279165 arielzoizner@mail.tau.ac.il) , Nadav Ravid (316317361 nadavravid1@mail.tau.ac.il) , Ido Moran 208436139 (idomorani1@mail.tau.ac.il), Ron Gothait 318939816 (rongothait@mail.tau.ac.il)

Background: Modern LLM-powered coding agents are highly effective at resolving isolated coding tasks, with state of the art models achieving a 90+% on coding evaluation frameworks like SWE-bench.

However, because these agents optimize for immediate functional correctness, they frequently generate code that is difficult to maintain - such as over-engineered solutions or localized, inelegant patches. Traditional benchmarks (SWE-bench, SWE-bench pro) do not measure these said maintainability issues. Recently introduced benchmarks, specifically ([SWE-CI, April 2026](#)), address this gap by evaluating long-term repository maintainability.

Hypothesis: Forcing an LLM coding agent to explicitly consider future, related tasks built on top of the current task will result in code that is more maintainable over time. This approach will improve performance on evolutionary benchmarks (SWE-CI) without significantly degrading success rates on traditional benchmarks (SWE-bench, SWE-bench pro).

Suggested Methodology: We propose a prompt-engineering framework that augments the standard coding pipeline. Given a primary issue or feature request, our system will:

1. Pass the input prompt and relevant codebase context to a smaller, cost effective model (e.g., Claude Haiku).
2. Generate a small set of synthetic future tasks (e.g., 3 feature extensions) that build upon the current requirement.
3. Synthesize these future tasks into an enhanced prompt for the primary coding agent.
4. Execute the coding task with the enhanced prompt.

Evaluation: We will evaluate our framework by comparing standard prompting against our future-task prompting framework.

- **Metrics:** We expect a non-to-negligible decrease in pass rates on traditional benchmarks and an increase in scores on evolutionary benchmarks (SWE-CI).
- **Variable Models:** We will test this across multiple model sizes (e.g., Haiku vs. Sonnet vs. Opus) to measure the impact on those.

Resource Requirements: Executing this evaluation requires API access to frontier models (OpenAI/Anthropic) to run the coding agents. Specifically, we require token allocations for:

1. Usage of a smaller model (e.g., Haiku) to generate the future-tasks within the pipeline.
2. Targeted usage of models of variable sizes to execute the actual coding tasks across the selected benchmarks evaluation tasks.